Helm Labs

helm-labs

Version 0.1.0

Table of Contents

1. Introduction 2
   1.1. How to use this book 2
   1.2. Build 2
2. CLI mechanics 3
   2.1. Core loop 3
   2.2. Inspect before you apply 3
   2.3. Session notes 3
3. Kustomize 4
   3.1. Mental model 4
   3.2. With Helm 4
   3.3. Session notes 4

Synthesized notes from hands-on Helm practice (CKA/CKAD, CGOA/CAPA, CKS). Lab drills stay in `chapters/`; this book captures what stuck.

Chapter 1. Introduction

This ebook is a side product of the lab repo, not a copy of the lab READMEs.

1.1. How to use this book

- Practice blind drills under `chapters/`.
- After a session, add a short AsciiDoc section here (traps, commands, cert notes).
- Rebuild HTML/PDF from `work/docs` (container build).

1.2. Build

Container-only (Podman or Docker). No host JDK required.

```
cd work/docs
./build.sh
# Windows: .\build.ps1
```

Outputs land in `generated/` (`book.html`, `book.pdf`).

Chapter 2. CLI mechanics

Cert focus: CKA, CKAD. Chart authoring is out of scope for the exam; release lifecycle is not.

2.1. Core loop

```
helm repo add … && helm repo update
helm install RELEASE CHART [-f values.yaml] [--set key=value]
helm upgrade RELEASE CHART …
helm history RELEASE
helm rollback RELEASE REVISION
helm uninstall RELEASE
```

2.2. Inspect before you apply

```
helm template RELEASE CHART --set …
helm install RELEASE CHART --dry-run --debug
helm get values RELEASE
helm get manifest RELEASE
```

2.3. Session notes

- Prefer `-f` for durable overrides; use `--set` for one-off tweaks (and know precedence).
- After a bad upgrade, read `helm history` then `helm rollback` — do not re-install blindly.

Chapter 3. Kustomize

Cert focus: CKA, CKAD, CGOA. Learn after Helm CLI, before deep chart authoring.

3.1. Mental model

- **Base** — shared manifests
- **Overlay** — env-specific patches, labels, replicas
- **Generators** — ConfigMap/Secret with content-hashed names

No template language. Patches and strategic merge instead.

3.2. With Helm

```
helm template RELEASE CHART -f values.yaml --output-dir chart-out
kubectl kustomize overlay   # resources point at rendered YAML
kubectl apply -k overlay
```

In Argo CD, pick **either** Helm **or** Kustomize as the Application source type.

3.3. Session notes